PROCESADORES SATELITALES

Integracion y Despliegue Continuo

Git · GitHub · GitHub Actions

ADAIP — 13 de abril de 2026

Contenidos

1. Evolucion del desarrollo de software
2. Que es la Integracion Continua?
3. Git — Control de versiones
4. GitHub — Trabajo en equipo
5. GitHub Actions — Automatizacion
6. Ejemplo practico — Procesador SAR
7. Caso de estudio completo — FastApi2
8. Resumen

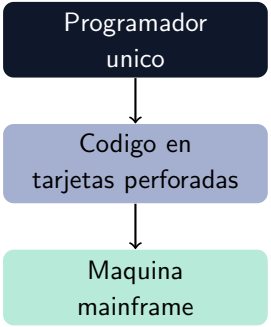
Los anos 60-70: el codigo como obra artesanal

Como se trabajaba

- El software lo escribia **una sola persona**
- No habia metodologia formal
- El codigo era considerado documentacion interna
- Los programas se entregaban en tarjetas perforadas
- No existia el concepto de "version"

El problema

Si el programador se iba, el conocimiento se iba con el. No habia forma de saber que habia cambiado ni por que.



Los anos 80: equipos y el caos de la colaboracion

Que cambio

- Los proyectos crecieron en complejidad
- Aparecieron los **equipos de desarrollo**
- Cada uno trabajaba en su propia copia del codigo
- La integracion se hacia al final del proyecto
- Aparecio el termino **"integration hell"**

Integration Hell

Cuando varios desarrolladores trabajaban meses en paralelo y al intentar unir el codigo, nada funcionaba. Los cambios de uno rompian el trabajo del otro. Resolver conflictos llevaba semanas.

Primera solucion: RCS y CVS (1982-1990)

Aparecieron los primeros sistemas de control de versiones. Un servidor central guardaba el historial. Solo una persona podia editar un archivo a la vez (**lock/unlock**).

Los anos 90: metodologias en cascada

1. **Requisitos** (meses)
2. **Diseno** (meses)
3. **Implementacion** (meses)
4. **Testing** (semanas)
5. **Deploy** (dias)

El problema

El cliente recibia el producto **anos despues** de haberlo pedido. Para entonces, los requisitos habian cambiado y el software ya era obsoleto.

Avance: Subversion (SVN) en 2000

Permitia que multiples personas editaran el mismo archivo. El servidor central seguia siendo el unico punto de verdad.

Los 2000: el Manifiesto Agil y el cambio de paradigma

Manifiesto Agil — 2001

17 desarrolladores se reunieron en Utah y firmaron un documento que cambiaria la industria:

- **Individuos e interacciones** sobre procesos y herramientas
- **Software funcionando** sobre documentacion extensiva
- **Colaboracion con el cliente** sobre negociacion contractual
- **Respuesta al cambio** sobre seguir un plan

Consecuencias practicas

- Ciclos cortos de entrega (**sprints** de 2 semanas)
- Feedback continuo del cliente
- Equipos pequenos y autonomos
- Testing como parte del desarrollo

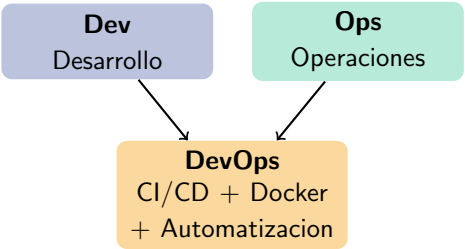
Git nace en 2005

Linus Torvalds crea Git para manejar el desarrollo del kernel de Linux. A diferencia de SVN, cada desarrollador tiene una copia completa del historial. No hay servidor central obligatorio.

Los 2010: DevOps y la fusion de desarrollo y operaciones

El problema anterior

- **Developers**: quieren cambiar cosas rapido
- **Operations**: quieren estabilidad, no cambios
- Resultado: conflicto permanente y deploys dolorosos
- *"Funciona en mi maquina"* era el lema



La solucion: DevOps

Unificar desarrollo y operaciones en un mismo equipo con las mismas herramientas y responsabilidades.

Herramientas clave

Docker (2013), GitHub Actions (2019), Kubernetes (2014)

La linea del tiempo completa

La conclusion

Cada etapa surgio como respuesta a los problemas de la anterior. CI/CD no es una moda — es la solucion natural a decadas de problemas de colaboracion, integracion y despliegue.

El problema que resuelve

Sin CI/CD

- Cada desarrollador trabaja aislado
- La integracion se hace al final
- Los errores aparecen tarde
- El deploy es un proceso manual y riesgoso
- *"Funciona en mi maquina"*

Con CI/CD

- El codigo se integra continuamente
- Los errores aparecen inmediatamente
- Los tests se corren automaticamente
- El deploy es automatico y reproducible
- El mismo codigo en todos los entornos

Definiciones

CI — Integracion Continua

Cada vez que un desarrollador sube codigo, se ejecuta automaticamente una bateria de tests para verificar que nada esta roto. Si algo falla, se notifica inmediatamente antes de que el problema se propague.

CD — Despliegue Continuo

Si los tests pasan, el codigo se despliega automaticamente al servidor correspondiente. Sin intervencion manual, sin pasos olvidados, siempre reproducible.

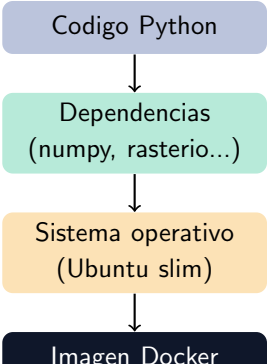
Docker — El entorno reproducible

El problema que resuelve

"Funciona en mi maquina" — el codigo corre perfecto en el entorno del desarrollador pero falla en produccion por diferencias en versiones de Python, librerias, o configuracion del sistema operativo.

Que es Docker

- Empaqueta la aplicacion junto con **todo su entorno**: Python, librerias, configuracion
- El paquete resultante se llama **imagen**
- Al ejecutar una imagen se crea un **contenedor**
- Corre igual en cualquier maquina



Que es Git?

Git es un sistema de **control de versiones distribuido**. Registra todos los cambios del codigo a lo largo del tiempo, permitiendo:

Historial completo

Cada cambio queda registrado con autor, fecha y mensaje

Ramas paralelas

Multiples lineas de desarrollo simultaneas

Reversibilidad

Volver a cualquier estado anterior del codigo

Conceptos clave

- **Repositorio**: carpeta con todo el historial del proyecto
- **Commit**: foto del estado del codigo en un momento dado
- **Rama (branch)**: linea de desarrollo independiente
- **Remote**: copia del repositorio en un servidor (GitHub)

Comandos fundamentales — Parte 1

git init

Crea un repositorio en la carpeta actual

```
git init                # repo nuevo
git init nombre-repo    # con nombre
```

git add

Prepara archivos para el proximo commit

```
git add archivo.py      # un archivo
git add .               # todos los
    cambios
git add src/            # una carpeta
```

git commit

Guarda los cambios con un mensaje

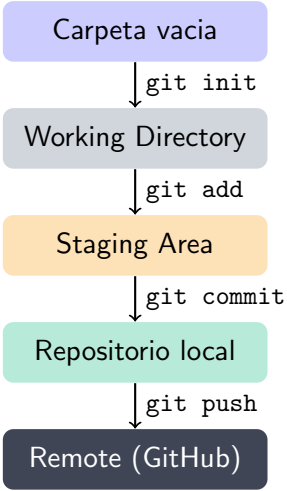
```
git commit -m "descripcion"
git commit --amend      # modificar el
    ultimo
```

git push

Sube commits al servidor remoto

```
git push origin main
git push origin develop
git push --force origin main
```

Flujo basico — de lo local al remoto



Comandos fundamentales — Parte 2

git pull

Descarga y fusiona cambios remotos

```
git pull origin main
git pull --rebase origin main
```

git fetch

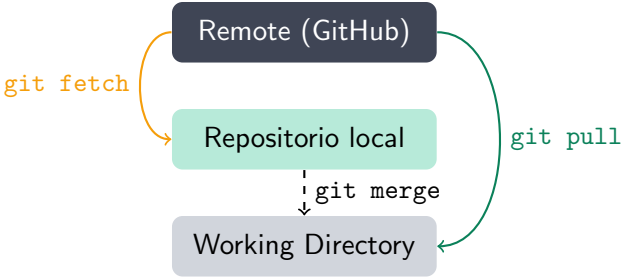
Descarga cambios SIN fusionarlos

```
git fetch origin
git fetch --all
```

fetch vs pull

- `pull` = `fetch` + `merge`
- `fetch` permite revisar antes de fusionar
- `fetch` es mas seguro en ramas compartidas

Flujo inverso — del remoto a lo local



Diferencia clave

`fetch` trae los cambios al repositorio local sin tocar tu codigo. `pull` los trae y los fusiona directamente en tu rama activa.

Comandos de ramas

git branch

Crea y lista ramas

```
git branch              # listar ramas
git branch nueva-rama   # crear rama
git branch -d rama      # eliminar
    rama
```

git status / log

```
git status
git log --oneline
git log --oneline --graph
git diff
```

git checkout / git switch

Cambia entre ramas

```
git checkout main
git checkout -b feature   # crear y
    cambiar
git switch develop
git switch -c nueva-rama
```

git stash

Guarda cambios temporalmente

```
git stash
git stash pop
git stash list
```

Tags — Versiones del proyecto

Los tags marcan puntos especificos en la historia, tipicamente versiones de lanzamiento.

Versionado semantico: MAJOR.MINOR.PATCH

v1.2.3

MAJOR: cambio que rompe compatibilidad
MINOR: funcionalidad nueva, compatible
PATCH: correccion de bugs

```
# Crear tag
git tag v1.0.0
git tag -a v1.0.0 -m "Primera
    version"

# Subir a GitHub
git push origin v1.0.0
git push origin --tags
```

```
# Ver tags
git tag
git tag -l "v1.*"

# Ir a un tag especifico
git checkout v1.0.0
```

GitHub vs Git

Git

- Herramienta de control de versiones
- Corre en tu maquina local
- No requiere internet
- Maneja el historial del codigo

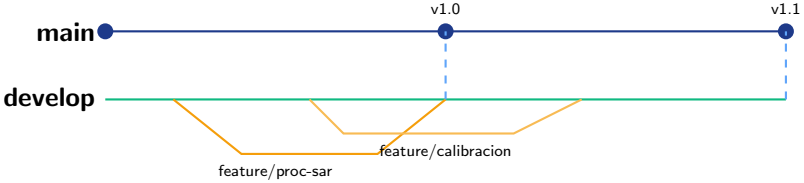
GitHub

- Plataforma web para alojar repositorios
- Permite colaboracion entre equipos
- Issues, Projects, Actions, Releases
- Alternativas: GitLab, Bitbucket

Analogia

Git es el auto. GitHub es la ruta por donde circula.

Estrategia de ramas para equipos



- **main**: codigo en produccion, siempre estable
- **develop**: integracion de funcionalidades, base para testing
- **feature/xxx**: desarrollo de funcionalidades individuales

Issues — Gestion de tareas

Los **Issues** son el sistema de GitHub para gestionar tareas, reportar errores, proponer mejoras y discutir cambios dentro de un repositorio.

Tipos de Issues

- **Bug**: algo no funciona correctamente
- **Feature**: nueva funcionalidad
- **Research**: investigacion tecnica
- **Docs**: documentacion pendiente

Buenas practicas

- Titulo claro y descriptivo
- Descripcion con contexto
- Asignar responsable
- Agregar etiquetas (labels)
- Vincular con un Milestone

Ejemplo para procesamiento SAR

#42 — Implementar correccion radiometrica para Sentinel-1
Aplicar LUT de calibracion segun documentacion ESA. Incluir test con escena de referencia de la Patagonia.

Milestones y Projects

Milestones — Hitos

Agrupan Issues relacionados con una meta o fecha limite.

Ejemplo:

- **v1.0 — Procesador basico**
  Issues #1, #2, #5, #8
- **v1.1 — Correccion atmosferica**
  Issues #9, #12, #15
- **v2.0 — Interfaz web**
  Issues #20, #21, #25

Projects — Tablero Kanban

Vista visual del estado de todas las tareas.

Backlog	En curso	Listo
#15	#8	#1
#20	#12	#2
#21		#5

Flujo recomendado

Issue → asignar a Milestone → mover en Project board → cerrar con PR

Pull Requests — Revision de codigo

Un **Pull Request (PR)** es una solicitud para fusionar una rama en otra.

1. Desarrollador crea rama `feature/mi-funcionalidad`
2. Trabaja y hace commits
3. Abre un PR hacia `develop`
4. Otro miembro del equipo revisa el codigo
5. Se discuten cambios, se hacen ajustes
6. Si los tests pasan y hay aprobacion $\rightarrow$ se fusiona

Ventajas

- Nadie sube codigo directamente a `main`
- Revision obligatoria antes de integrar
- Historial de decisiones tecnicas documentado
- Los tests corren automaticamente antes de aprobar

Que son los Workflows?

Un **workflow** es un proceso automatizado definido en un archivo YAML dentro del repositorio.

Caracteristicas

- Viven en `.github/workflows/nombre.yml`
- Se activan por eventos: push, pull request, horario, manual
- Corren en maquinas virtuales de GitHub (runners)
- Son codigo: se versionan junto al proyecto

Disparador

push, pull request, schedule

Jobs

test, build, deploy

Steps

comandos individuales

Anatomia de un workflow

```yaml
name: CI/CD                        # nombre del workflow

on:                                # cuando se ejecuta
  push:
    branches: [main, develop]      # en push a estas ramas
  pull_request:
    branches: [main]               # en PR hacia main

jobs:                              # trabajos a ejecutar

  test:                            # ID del job
    name: Correr tests             # nombre visible en GitHub
    runs-on: ubuntu-latest         # sistema operativo del runner
    steps:
      - name: Descargar codigo
        uses: actions/checkout@v4

      - name: Instalar Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.13"

      - name: Correr tests
        run: pytest tests/ -v
```

Pipeline completo: test → build → deploy

```yaml
jobs:

  test:                           # Job 1: tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest tests/ -v

  build:                          # Job 2: solo si test paso
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ghcr.io/mi-usuario/mi-app:latest

  deploy:                         # Job 3: solo en main
    needs: [test, build]
    if: github.ref == 'refs/heads/main'
    steps:
      - run: curl -X POST "${{ secrets.DEPLOY_HOOK }}"
```

Conceptos clave de los workflows

Disparadores (on:)

- `push` — al subir codigo
- `pull_request` — al abrir un PR
- `schedule` — con cron (periodico)
- `workflow_dispatch` — manual
- `release` — al publicar una version

Contextos disponibles

- `github.ref` — rama actual
- `github.actor` — quien hizo push
- `github.repository` — usuario/repo
- `github.sha` — hash del commit

Secrets

Variables sensibles guardadas en GitHub Settings. Se referencian como ${{ `secrets.NOMBRE` }} y nunca aparecen en los logs.

needs — Dependencias

`needs: test` hace que un job espere a que otro termine exitosamente antes de ejecutarse.

Caso de uso: procesador SAR con CI/CD

Escenario

Equipo de 3 personas desarrollando un procesador de imagenes Sentinel-1 en Python. Necesitan garantizar que cada cambio no rompa el procesamiento existente.

Estructura del proyecto

- `processor/` — algoritmos SAR
- `tests/` — tests con imagenes de referencia
- `data/` — escenas de calibracion
- `Dockerfile` — entorno reproducible
- `.github/workflows/`

Flujo de trabajo

1. Issue #42: nueva funcion de filtrado
2. Rama `feature/filtro-lee`
3. Desarrollo + tests unitarios
4. PR hacia `develop`
5. CI corre tests automaticamente
6. Revision del equipo
7. Merge → deploy a staging
8. Validacion → merge a `main`

Workflow para procesamiento satelital

```yaml
name: CI --- Procesador SAR

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [develop]

jobs:

  test:
    name: Tests de procesamiento
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Instalar dependencias
        run: |
          pip install numpy scipy rasterio pytest
          pip install -r requirements.txt

      - name: Tests unitarios
        run: pytest tests/unit/ -v

      - name: Tests con escenas de referencia
        run: pytest tests/integration/ -v --timeout=120

      - name: Calidad del codigo
        run: |
          flake8 processor/
          black processor/ --check
```

Beneficios concretos para el equipo

Sin CI/CD

- El procesador SAR funciona solo en la PC del Dr. Garcia
- Al pasar a produccion falla por version diferente de GDAL
- Nadie sabe que cambio rompio el coregistro
- El deploy tarda 2 dias y requiere intervencion manual

Con CI/CD

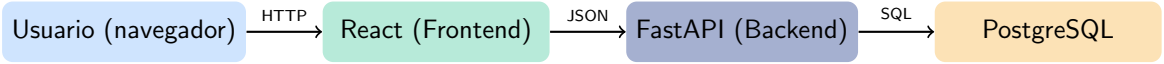
- Docker garantiza el mismo entorno siempre
- Si un cambio rompe el procesamiento, se detecta en minutos
- El historial de Git muestra exactamente que cambio
- El deploy es automatico y tarda 5 minutos

Principio fundamental

El codigo que paso los tests en CI es exactamente el mismo que se despliega en produccion.

Arquitectura del sistema FastApi2

Sistema web completo con CI/CD, compuesto por dos repositorios independientes.



Repositorio backend

`sebaheredia/FastApi2`

- `main.py` — endpoints REST
- `database.py` — conexion a PostgreSQL
- `models.py` — tabla users (SQLAlchemy)
- `schemas.py` — validacion JSON (Pydantic)
- `conftest.py` — setup de tests

Repositorio frontend

`sebaheredia/fastapi2-frontend`

- `src/App.js` — componente React principal
- `src/api.js` — llamadas al backend
- `src/App.css` — estilos
- `nginx.conf` — servidor web

Docker — Backend

El backend usa un Dockerfile de una sola etapa. Crea las tablas en PostgreSQL antes de arrancar el servidor:

```
FROM python:3.13-slim
WORKDIR /app

# Copiar deps primero para aprovechar cache de Docker
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8000

# Dos comandos en secuencia con &&:
# 1. Crear tablas en PostgreSQL (create_all)
# 2. Solo si tuvo exito, arrancar uvicorn
CMD ["sh", "-c",
  "python -c 'from database import engine; import models;
   models.Base.metadata.create_all(bind=engine)'
   && uvicorn main:app --host 0.0.0.0 --port $PORT"]
```

Por que psycopg2-binary es critico

Sin este driver en `requirements.txt`, la imagen se buildea sin soporte para PostgreSQL y la app cae silenciosamente a SQLite aunque `DATABASE_URL` este correctamente configurada.

Docker — Frontend (multi-stage)

El frontend usa un build de dos etapas: Node compila React y nginx sirve los archivos estaticos:

```
# Etapa 1: Node compila React
FROM node:20-slim AS builder
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .

# La URL del backend se embebe en el bundle en tiempo de build
ARG REACT_APP_API_URL
ENV REACT_APP_API_URL=$REACT_APP_API_URL
RUN npm run build     # genera archivos estaticos en /build

# Etapa 2: nginx sirve los archivos estaticos
FROM nginx:alpine
COPY --from=builder /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Ventaja del multi-stage

La imagen final pesa ∼25 MB (nginx) en lugar de ∼300 MB (Node). Node solo se usa para compilar, no para servir.

Base de datos PostgreSQL en Render

SQLite dentro del Docker (incorrecto)

- Datos guardados en un archivo dentro del contenedor
- Al redesplegar, Render destruye el contenedor
- El archivo desaparece con el contenedor
- Todos los datos se pierden en cada deploy

PostgreSQL externo (correcto)

- Servidor independiente en Render
- El contenedor puede destruirse y recrearse
- La DB nunca se toca en los deploys
- Los datos persisten siempre

Conexion via variable de entorno

Render inyecta `DATABASE_URL` dentro del contenedor al arrancarlo. El codigo la lee con `os.getenv()`:

```
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./database.db")
```

Si la variable existe → usa PostgreSQL. Si no → usa SQLite (desarrollo local).

GHCR — Registro de imagenes Docker

GitHub Container Registry almacena las imagenes construidas por GitHub Actions. Render las descarga directamente.

Imagenes del backend

- `ghcr.io/sebaheredia/fastapi2:main`
- `ghcr.io/sebaheredia/fastapi2:develo`

Imagenes del frontend

- `ghcr.io/sebaheredia/fastapi2-fronte`
- `ghcr.io/sebaheredia/fastapi2-fronte`

Ventaja clave

La imagen que corre en produccion es exactamente la misma que fue testeada en CI. No hay diferencias de entorno.

Pipeline CI/CD del backend

```yaml
name: CI/CD Backend
on:
  push:
    branches: [main, develop]

jobs:
  test:                              # Job 1: tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.13"}
      - run: pip install -r requirements.txt
      - run: pytest test_main.py -v  # si falla, se detiene

  docker:                            # Job 2: build imagen
    needs: test
    steps:
      - uses: docker/login-action@v3
        with: {registry: ghcr.io, ...}
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ghcr.io/sebaheredia/fastapi2:main

  deploy:                            # Job 3: notificar a Render
    needs: docker
    if: github.ref == 'refs/heads/main'
    run: |
      curl -X PATCH "api.render.com/v1/services/.../image" ...
      curl -X POST "api.render.com/v1/services/.../deploys" ...
```

Pipeline CI/CD del frontend

El frontend tiene una particularidad: `REACT_APP_API_URL` debe determinarse antes del build.

```
jobs:
  docker:
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with: {registry: ghcr.io, ...}

      # Determinar URL del backend segun la rama
      - name: Definir URL del backend
        id: backend_url
        run: |
          if [ "${{ github.ref }}" = "refs/heads/main" ]; then
            echo "url=${{ secrets.BACKEND_PRODUCTION_URL }}" >> $GITHUB_OUTPUT
          else
            echo "url=${{ secrets.BACKEND_STAGING_URL }}" >> $GITHUB_OUTPUT
          fi

      # La URL se embebe en el bundle de React durante el build
      - uses: docker/build-push-action@v5
        with:
          build-args: |
            REACT_APP_API_URL=${{ steps.backend_url.outputs.url }}
          push: true
          tags: ghcr.io/sebaheredia/fastapi2-frontend:main
```

Servicios desplegados en Render

4 servicios web + 1 base de datos

Servicio	Tipo	Rama	URL
fastapi2-staging	Image/GHCR	develop	...staging-docker.onrender.com
fastapi2-production	Image/GHCR	main	...production-docker.onrender.com
fastapi2-frontend-staging	Image/GHCR	develop	...frontend-staging.onrender.com
fastapi2-frontend-main	Image/GHCR	main	...frontend-main.onrender.com
fastapi2-db	PostgreSQL	—	red interna de Render

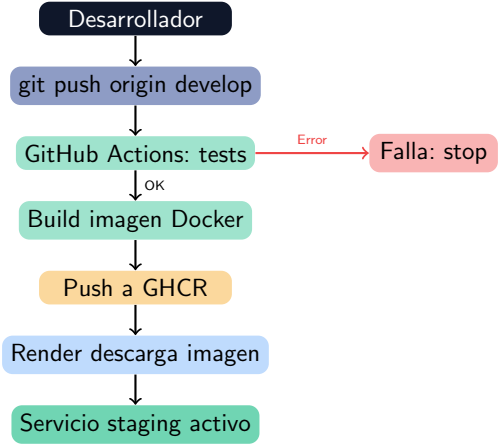
Secrets en GitHub (backend)

- `RENDER_API_KEY`
- `RENDER_STAGING_SERVICE_ID`
- `RENDER_PRODUCTION_SERVICE_ID`

Secrets en GitHub (frontend)

- `RENDER_API_KEY`
- `RENDER_FRONTEND_STAGING_SERVICE_ID`
- `RENDER_FRONTEND_PRODUCTION_SERVICE_I`

Flujo completo de un cambio en produccion



CORS — Comunicacion entre dominios

El problema

Frontend (`fastapi2-frontend.onrender.com`) y backend (`fastapi2-backend.onrender.com`) estan en dominios distintos. El navegador bloquea los requests entre dominios por seguridad.

La solucion: middleware CORS en FastAPI

```
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
  CORSMiddleware,
  allow_origins=["*"],
  allow_methods=["*"],
  allow_headers=["*"],
)
```

Importante

Resumen

Lo que vimos hoy

1. **Git**: control de versiones
   - add, commit, push, pull, fetch
   - merge, checkout, stash
   - tags y versionado semantico
2. **GitHub**: plataforma colaborativa
   - Issues, Milestones, Projects
   - Ramas: main, develop, feature
   - Pull Requests
3. **GitHub Actions**
   - Workflows, jobs, steps
   - Disparadores y condiciones
   - Secrets y contextos

Proximos pasos

1. Crear repositorio para el procesador SAR
2. Configurar ramas main/develop
3. Escribir primeros tests automatizados
4. Crear el primer workflow de CI
5. Agregar Docker para reproducibilidad
6. Configurar deploy automatico

Herramientas necesarias

Git, VSCode, Docker Desktop, cuenta en GitHub

Gracias!

ADAIP

Area de Desarrollos Avanzados de Imagenes y Percepcion

Repositorios del ejemplo practico:

github.com/sebaheredia/FastApi2

github.com/sebaheredia/fastapi2-frontend